

Zeus Engine

Team 08

국동진 김기훈 이준협 정찬일

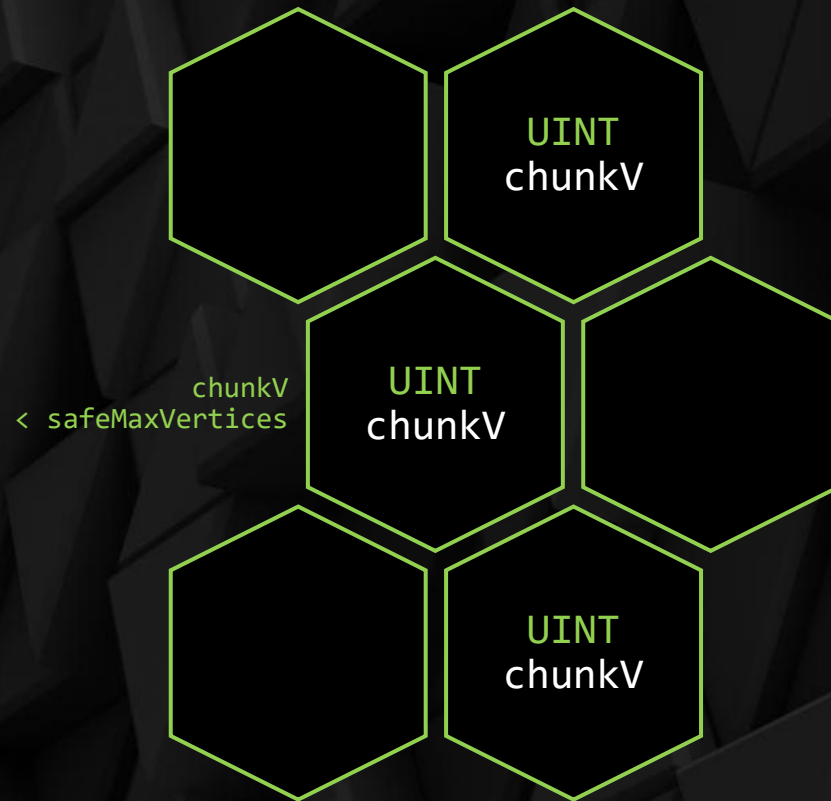
01 Engine Core

문제 01

VertexBuffer가 넘친다.

한번에 보낼 수 있는 정점 데이터가 버퍼의 MaxVertices를 초과할 경우
데이터 손실 발생.

버퍼를 더 늘리자고? 뻥센데...



무작정 버퍼를 늘리는 데에는 한계가 있다.

>> 1번에 끝낼 Draw Call을 여러 번 나눠서 처리한다.

// 비용이 좀 들긴 하지만 안정성이 증가.

예를 들어, ByteWidth = 64 KB

>> 정점 하나가 32 B일 경우 한 chunk에 약 2,048개.

while (chunkStart < totalVertices)

```
UINT chunkStart = 0;
while (chunkStart < totalVertices)
{
    const UINT remaining = totalVertices - chunkStart;
    const UINT chunkV = (std::min)(safeMaxVertices, remaining);
    const UINT chunkI = (chunkV / 4) * 6;
    const UINT indexStart = (chunkStart / 4) * 6;
```

```
std::memcpy(
    mapped.pData,
    Vertices.data() + chunkStart,
    sizeof(FVertexFont) * chunkV
);
Context->Unmap(Geometry->VertexBuffer.Get(), 0);
```

```
Context->DrawIndexed(chunkI, 0, 0);
drewAny = true;

chunkStart += chunkV;
```

chunkStart = 0;

// 0번째 정점부터 시작

Const UINT chunkV

// 이번에 보낼 정점 데이터의 양

Map / Unmap

// chunkV만큼의 데이터를 GPU로 복사

Context->DrawIndexed()

// Draw Call -> 반복 ...

02 Editor & Rendering

문제 02-1

모든 한글 조합은 총 11,172자.

알파벳과 달리 한글은 자음과 모음의 위치에 따라 글자의 모양이 변함. 모든 조합을 미리
굽기에는 VRAM이 부족하고, font 파일 전체를 로드하는 것은 비효율적.

실시간으로 필요한 글자만 생성하자. >> **Dynamic Atlas**

수만 개의 한글 조합을 미리 다 그리지 않고, 필요한 순간에만 즉석에서 구워 낸다.

>> **On-demand Generation**

게으른 생성... >> `bool EnsureGlyph()`

```
bool FDynamicFontAtlas::EnsureGlyph(uint32 codepoint)
{
    if (GlyphMap.Regions.find(codepoint) != GlyphMap.Regions.end())
        return true;

    FAtlasRegion region = {};
    if (!RasterizeGlyph(codepoint, region, AtlasPixels, AtlasWidth, AtlasHeight))
        return false;

    GlyphMap.Regions[codepoint] = region;
    UploadRect(region.x, region.y, region.width, region.height);
    return true;
}
```

11,172자를 미리 로드하지 않고, 출력 직전에만 글자가 있는지 체크.

글자가 이미 있으면

>> `return true;` // 통과

없으면

>> `return false;` // 즉석에서 비트맵 생성

VRAM 안에 글자들 다닥다닥 넣기... >> `bool AllocateRect()`

```
bool FDynamicFontAtlas::AllocateRect(int w, int h, int& outX, int& outY)
{
    const int Padding = 1;

    w += Padding * 2;
    h += Padding * 2;

    if (PenX + w > AtlasWidth)
    {
        PenX = 0;
        PenY += ShelfHeight;
        ShelfHeight = 0;
    }

    if (PenY + h > AtlasHeight)
        return false;

    outX = PenX + Padding;
    outY = PenY + Padding;

    PenX += w;
    ShelfHeight = (std::max)(ShelfHeight, h);
    return true;
}
```

>> SHELF PACKING ALGORITHM

>> 글자를 한 줄씩 쌓다가 너비(AtlasWidth)가
꽤 차면, 현재 층의 최대 높이(ShelfHeight)를
기준으로 새로운 행을 할당하는 방식.

>> ... // 빈틈없이 공간 활용!

문제 02-2

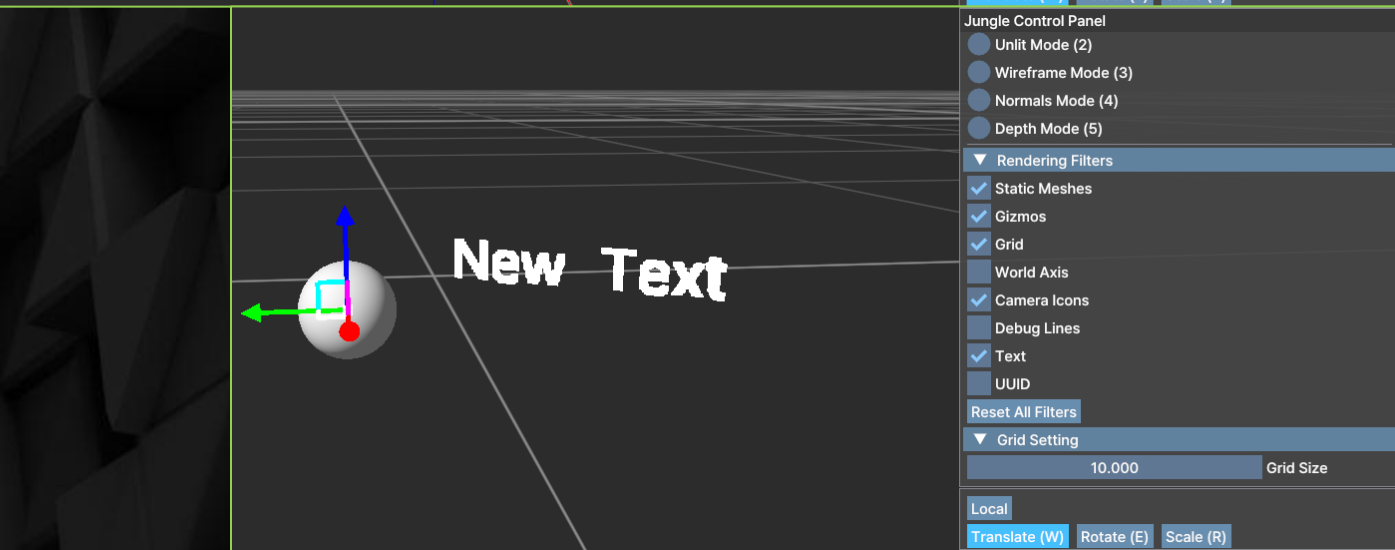
에디터에서 원하는 object만 보거나
Polygon, Lighting 상태 확인이 어렵다.

사실 어렵지 않은 문제. ShowFlag와 ViewMode를 아직 구현하지 못해서 그랬을 뿐.

Rendering Filters

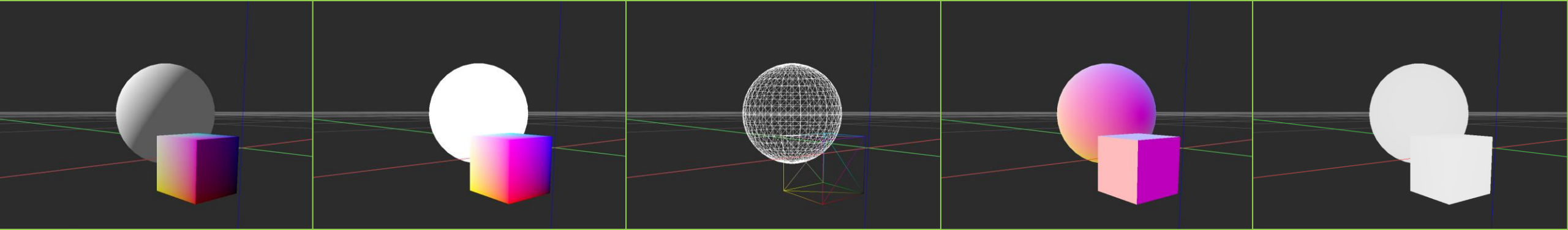


>> Rendering Filters 옵션을 모두 켜었을 경우. Text의 경우 별도의 Object로 생성 가능.



>> Rendering Filters 옵션을 일부만 켜었을 경우. Static Meshes 옵션을 끌 경우 Picking이 불가능하다.

View Mode



Lit Mode (1)

>> 모든 Material
과 Lighting이 적
용된 Scene의 최종
결과를 보여주는
Mode.

Unlit Mode (2)

>> 모든 Lighting을
제거한 Base Color
Only Mode.

Wireframe Mode (3)

>> Scene의 모든
Polygon edge들을
보여주는 Mode.

Normals Mode (4)

>> 모든 Object들의
표면의 Normal
Vector의 방향을 색
깔로 보여주는 Mode.

Depth Mode (5)

>> Scene의 Depth를
흑색에서 백색으로
Gradient하게 보여
주는 Mode.

Showcase